

Administración y Diseño de Bases de Datos

Apuntes de Teoría y Práctica

Daniel Garcia Salinas

28 de enero de 2026

Índice general

0.1.	Tema 1: Introducción a la Administración de Bases de Datos	2
0.1.1.	Instalación	2
0.1.2.	Estructura del Diccionario de Datos	2
0.1.3.	Motores de Almacenamiento	2
0.1.4.	Archivos de Log en MySQL	3
0.1.5.	GUIÓN PRÁCTICA 1:	3
0.2.	Tema 2: Gestión de transacciones	5
0.2.1.	TRANSACCIONES EN INNODB	5
0.2.2.	GUIÓN PRÁCTICA 2:	5
0.3.	Procedimientos almacenados	11
0.3.1.	PROCEDIMIENTOS ALMACENADOS	11
0.3.2.	VARIABLES	11
0.3.3.	CONTROL DE FLUJO	11
0.3.4.	GUIÓN PRÁCTICA 3:	13
0.3.5.	BLOQUE 2 - EJERCICIO TEMA 2:	14
0.4.	Tema 3: Control de concurrencia	14
0.4.1.	Bloqueos	14
0.4.2.	GUIÓN PRÁCTICA 4:	15
0.4.3.	Ejercicio: Análisis de Bloqueos en el Motor Transaccional	15
0.4.4.	Ejercicio: Conflicto de Bloqueos y Lock Wait Timeout	17
0.5.	Tema 4: Recuperación de la base de datos	19
0.5.1.	Copias de seguridad:	19
0.5.2.	GUIÓN PRÁCTICA 5:	20
0.5.3.	Plan de copias de seguridad de una empresa (supuesto)	21
0.6.	Tema 5: Seguridad y privacidad en bases de datos	22
0.6.1.	Creación de usuarios	22
0.6.2.	Concesión de permisos	23
0.6.3.	ROLES	23
0.6.4.	BLOQUEO DE CUENTAS DE USUARIO	24
0.6.5.	ACTIVAR FICHEROS DE LOG PARA AUDITORÍA	24
0.6.6.	CIFRADO DE DATOS	24
0.6.7.	GUIÓN PRACTICA 6:	24
0.6.8.	GUIÓN PRÁCTICA 7:	24
0.6.9.	Conexiones desde el exterior	25

0.1. Tema 1: Introducción a la Administración de Bases de Datos

0.1.1. Instalación

Terminal

```
sudo apt install mysql-server
```

0.1.2. Estructura del Diccionario de Datos

Directorio	Base de datos	Descripción
mysql	mysql	Esquemas MySQL. Requerida para la ejecución del servidor. Contiene tablas del diccionario de datos y del sistema, con información sobre <i>grants</i> , ficheros <i>log</i> , del optimizador, etc.
performance_schema	performance_schema	Contiene información útil para inspeccionar la ejecución del servidor en tiempo de ejecución.
sys	sys	Ayuda a interpretar la información de <i>Performance Schema</i> .
–	Information_schema	Proporciona acceso al diccionario de datos o catálogo del sistema. Es una base de datos estándar, pero que no se implementa sobre un directorio específico.

Tabla 1: Bases de datos internas de MySQL.

0.1.3. Motores de Almacenamiento

Para visualizar los motores de almacenamiento disponibles en el servidor:

```
SHOW engines;
```

O bien, realizar una consulta directa al diccionario de datos a través de la tabla `engines` en el esquema `information_schema`:

```
SELECT engine FROM information_schema.engines;
```

Para consultar información sobre metadatos:

```
SELECT table_name, table_schema FROM INFORMATION_SCHEMA.TABLES;
```

Es fundamental conocer las rutas donde el servidor almacena su configuración y sus datos:

- **Archivo de configuración:** `/etc/mysql/my.cnf` y `/etc/mysql/my.conf.d/mysqld.cnf`
- **Directorio de almacenamiento:** `/var/lib/mysql`
- **Inicio, parada y reinicio:** `sudo service mysql start/stop/restart`

Si necesitamos consultar los valores que una instancia está utilizando en tiempo de ejecución:

```
SHOW variables;
```

Si queremos ver los usuarios existentes en el sistema:

```
SELECT user, host from mysql.user;
```

Creación de un nuevo usuario:

```
CREATE USER 'sammy'@'localhost' IDENTIFIED BY 'password';
```

Cambiar el password:

```
ALTER USER user() IDENTIFIED BY 'newPass';
```

0.1.4. Archivos de Log en MySQL

Tipo de Log	Descripción
Error log	Registro de errores. Indica cuándo arrancó y se detuvo el servidor. Se graba por defecto en la carpeta de datos de MySQL (archivo <code>host_name.err</code>), aunque la variable <code>log_error</code> permite indicar otra ruta.
General Query log	Registro general de consultas. Registra las conexiones y consultas enviadas por los clientes al servidor.
Binary log	Registro de las operaciones que han modificado datos. Es crucial para procesos de replicación y recuperación.
Relay log	Cambios de datos recibidos desde un servidor replicado.
Slow query log	Consultas que requirieron más tiempo del especificado en la variable <code>long_query_time</code> .
DDL log (metadata log)	Operaciones sobre metadatos causadas por sentencias DDL (<i>Data Definition Language</i>).

Tabla 2: Tipos de archivos de registro en MySQL.

0.1.5. GUION PRÁCTICA 1:

1. Versión: `mysql --version`

2. Acceso usando el usuario por defecto: `mysql -u root` - Por defecto el usuario root no tiene password. El sistema de instalación asigna un método de autenticación llamado: `auth_socket`

3. En que base de datos se encuentra la información sobre el resto de bases de datos: `information_schema.SCHEMATA`

4. Motores disponibles:

```
ALTER USER user() IDENTIFIED BY 'newPass';
```

En la columna Transactions si hay alguno que pone YES es transaccional

5. `information_schema`: tabla que sirve para guardar metadatos del servidor. InnoDB es el motor asociado por defecto.

6. Revisar usuarios creados:

```
SELECT user, host, plugin FROM mysql.user;
```

7. Creación de usuarios

■ En local:

```
CREATE USER 'user_local'@'localhost'  
IDENTIFIED BY 'Password123!';
```

■ En remoto:

```
CREATE USER 'user_remote'@'%' IDENTIFIED BY '  
Password123!';
```

8. Registro de errores `/var/log/mysql/error.log`

0.2. Tema 2: Gestión de transacciones

0.2.1. TRANSACCIONES EN INNODB

- Inicio de una transacción: `START TRANSACTION|BEGIN`
- Fin de una transacción: `COMMIT|ROLLBACK`
- Cambio del modo de commit automático:

```
SET AUTOCOMMIT = 0;
```

También se puede cambiar en el fichero de configuración

0.2.2. GUIÓN PRÁCTICA 2:

- SECCIÓN 1: ver motores y ver el por defecto

```
mysql> show variables like 'default_storage_engine';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| default_storage_engine | InnoDB |
+-----+-----+
1 row in set (0,00 sec)
```

- TRANSACCIÓN 1: BEGIN + ROLLBACK

```
-- Crear la base de datos para la practica
CREATE DATABASE PR1;

-- Seleccionar la base de datos de trabajo
USE PR1;

-- Crear tabla con clave primaria
CREATE TABLE ejemplo (
    campo INT PRIMARY KEY
);

-- 1. Insertar filas iniciales (Autocommit activo)
INSERT INTO ejemplo VALUES (1);
INSERT INTO ejemplo VALUES (2);
INSERT INTO ejemplo VALUES (3);

-- 2. Iniciar una transaccion manual
BEGIN;

-- 3. Insertar nueva fila dentro de la transaccion
INSERT INTO ejemplo VALUES (4);

-- 4. Comprobar estado temporal (se verán 4 filas)
SELECT * FROM ejemplo;
```

```
-- 5. Ejecutar la sentencia de reversion
ROLLBACK;

-- 6. Verificacion final (la fila 4 ha desaparecido)
SELECT * FROM ejemplo;
```

Resultado

Tras ejecutar el comando ROLLBACK, el motor InnoDB devuelve la tabla al estado exacto en el que estaba antes del BEGIN. Esto causa que la inserción del valor 4 nunca llegue a ser persistente.

■ TRANSACCIÓN 2: BEGIN + EXIT

```
-- 1. Iniciar una nueva transaccion manual
BEGIN;

-- 2. Insertar fila con el valor 4
INSERT INTO ejemplo VALUES (4);

-- 3. Comprobar estado temporal (se visualizan 4 filas)
SELECT * FROM ejemplo;

-- 4. Finalizar la sesion sin ejecutar COMMIT ni ROLLBACK
EXIT;
```

Tras reconectar al servidor y seleccionar la base de datos:

```
-- 5. Reconectar y verificar contenido
USE PR1;
SELECT * FROM ejemplo;
```

Resultado

Al reconectar, se observa que solo existen las filas con los valores **1, 2 y 3**. La fila con el valor **4** ha desaparecido.

Justificación: Al cerrar la sesión (sentencia EXIT) o perder la conexión mientras una transacción está abierta (sin haber ejecutado COMMIT), MySQL realiza un **ROLLBACK**.

■ TRANSACCIÓN 3: BEGIN + COMMIT + EXIT

```
-- 1. Iniciar una nueva transaccion manual
BEGIN;

-- 2. Insertar fila con el valor 4
INSERT INTO ejemplo VALUES (4);

-- 3. Comprobar estado temporal (se visualizan 4 filas)
SELECT * FROM ejemplo;

-- 4. Confirmar los cambios de forma permanente
COMMIT;

-- 5. Finalizar la sesion
EXIT;
```

Tras reconectar al servidor y seleccionar la base de datos:

```
-- 6. Reconectar y verificar contenido
USE PR1;
SELECT * FROM ejemplo;
```

Resultado

Al reconectar, se observa que la tabla contiene las **4 filas** (1, 2, 3 y 4). El valor 4 se ha mantenido de forma persistente.

La sentencia **COMMIT** finaliza la transacción actual y hace que todos los cambios realizados durante la misma se vuelvan permanentes y visibles para otros usuarios.

■ TRANSACCIÓN 4: Concurrency and Isolation between sessions

En este escenario se analiza cómo interactúan dos usuarios que acceden simultáneamente a los mismos datos. El orden cronológico de las operaciones es el siguiente:

Paso 1: Sesión 1 - Inicio de Transacción:

```
BEGIN;  
INSERT INTO ejemplo VALUES (5);
```

Paso 2: Sesión 2 - Consulta de datos (Lectura Sucia):

```
-- Se ejecuta mientras la Sesión 1 sigue abierta  
SELECT * FROM ejemplo;
```

Resultado

En la Sesión 2 solo se ven las filas **1, 2, 3 y 4**. El valor **5** no aparece, ya que aun no se realizó *COMMIT*.

Paso 3: Sesión 1 - Consulta de datos propios:

```
SELECT * FROM ejemplo;
```

Resultado

En la Sesión 1 se ven las **5 filas**, ya que ve toda la información relativa a su espacio de trabajo actual.

Paso 4: Sesión 1 - Confirmación:

```
COMMIT;
```

Paso 5: Sesión 2 - Consulta final:

```
SELECT * FROM ejemplo;
```

Resultado Final

Tras el *COMMIT* de la primera sesión, la Sesión 2 ahora sí puede visualizar las **5 filas**.

■ TRANSACCIÓN 5: Uso de la variable AUTOCOMMIT

```
-- 1. Creacion de la tabla de pruebas
CREATE TABLE CLIENTE (
    A INT PRIMARY KEY,
    B CHAR(20)
);

-- 2. Desactivar el modo de confirmacion automatica
SET AUTOCOMMIT = 0;

-- 3. Insercion de datos
INSERT INTO CLIENTE VALUES (10, 'Juan');
INSERT INTO CLIENTE VALUES (15, 'Maria');

-- 4. Reversion de los cambios
ROLLBACK;

-- 5. Consulta de verificacion
SELECT * FROM CLIENTE;
```

Resultado

Tras realizar el *SELECT*, obtenemos un conjunto vacío, esto ocurre porque la variable *AUTOCOMMIT* desactiva que las transacciones se realicen de forma automática. Al hacer *ROLLBACK* se deshacen los cambios realizados.

0.3. Procedimientos almacenados

0.3.1. PROCEDIMIENTOS ALMACENADOS

Creación de una PROCEDURE:

```
CREATE PROCEDURE nombre ([parametro[,...]])  
[opciones] codigo_sql
```

Creación de una FUNCTION:

```
CREATE FUNCTION nombre ([parametro[,...]]) RETURNS tipo_de_datos  
[opciones] codigo_sql
```

Borrarlas:

```
DELETE FUNCTION/PROCEDURE [IF EXISTS] nombre
```

Modificarlas:

```
ALTER FUNCTION/PROCEDURE nombre [NAME nombre\_nuevo][SQL SECURITY  
DEFINER/INVOKER][COMMENT 'nuevo\_comentario']
```

Llamar a una función: (siendo acortar() la función)

```
SELECT acortar(titulo, 30), acortar(titulo, 20) FROM titulos ...
```

Llamar a un procedimiento: (siendo obtener_titulo el procedimiento)

```
CALL obtener_titulo(1,@resultado);  
SELECT @resultado;
```

0.3.2. VARIABLES

```
DECLARE nombre_var1 <tipo_datos> [DEFAULT valor];  
SET var1=valor1  
SELECT var:=valor  
SELECT ___ INTO var
```

0.3.3. CONTROL DE FLUJO

IF-THE-ELSE:

```
IF condicion THEN  
  instrucciones;  
[ELSE IF condicion THEN  
  instrucciones;]  
[ELSE  
  instrucciones;]  
END IF;
```

WHILE-DO:

```
[nombre_bucle:] WHILE condicion DO  
    instrucciones;  
END WHILE [nombre-.bucle];
```

LOOP:

```
nombre_bucle: LOOP  
    instrucciones;  
END LOOP nombre_loop;
```

0.3.4. GUION PRÁCTICA 3:

Directorio seguro para importar el .csv `/var/lib/mysql-files/`

■ Procedimiento de clasificación de países:

```
-- Cambio del delimitador para permitir el uso de ';' internamente
DELIMITER //

CREATE PROCEDURE clasificar_pais (
    IN pais VARCHAR(100),
    OUT clasificacion VARCHAR(100)
)
BEGIN
    DECLARE densidad_val INT;

    -- Obtener la densidad del pais indicado
    SELECT Density INTO densidad_val
    FROM COUNTRIES
    WHERE country LIKE CONCAT('%', pais, '%')
    LIMIT 1;

    -- Logica de clasificacion segun el guion
    IF densidad_val > 500 THEN
        SET clasificacion = '1';
    ELSEIF densidad_val > 100 AND densidad_val <= 500 THEN
        SET clasificacion = '2';
    ELSE
        SET clasificacion = '3';
    END IF;
END //

DELIMITER ;
```

■ Ejecución

```
-- Caso 1: Pais con densidad nivel 3 (Baja)
CALL clasificar_pais('Venezuela', @resultado);
SELECT @resultado; -- Devuelve '3'

-- Caso 2: Pais con densidad nivel 1 (Muy Alta)
CALL clasificar_pais('Bahrain', @resultado);
SELECT @resultado; -- Devuelve '1'
```

■ Ejercicio aviso si densidad menor que umbral

```
DELIMITER //

CREATE PROCEDURE aviso (
    IN pais VARCHAR(100),
    OUT mensaje VARCHAR(100)
```

```

)
BEGIN
    DECLARE densidad_val INT;

    -- Búsqueda de la densidad usando coincidencia parcial
    SELECT Density INTO densidad_val
    FROM COUNTRIES
    WHERE country LIKE CONCAT('%', pais, '%')
    LIMIT 1;

    -- Evaluación de umbral crítico
    IF densidad_val < 10 OR densidad_val IS NULL THEN
        SET mensaje = 'La densidad es menor que 10 o el país
                        es inexistente';
    ELSE
        SET mensaje = 'todo bien';
    END IF;
END //

DELIMITER ;

```

0.3.5. BLOQUE 2 - EJERCICIO TEMA 2:

Listing 1: Implementación de DDL con Foreign Keys

```

-- 1. Crear tabla principal de usuarios
CREATE TABLE user (
    userid INT,
    last_login DATE,
    registration DATE,
    PRIMARY KEY (userid)
);

-- 2. Crear tabla de datos extendidos con clave foránea
CREATE TABLE user_data (
    userid INT,
    email VARCHAR(20),
    name VARCHAR(100),
    contact INT,
    obsv VARCHAR(500),
    FOREIGN KEY (userid) REFERENCES user(userid)
);

```

0.4. Tema 3: Control de concurrencia

0.4.1. Bloqueos

- **SELECT ... FOR UPDATE:** Bloquea las filas para que otros procesos no puedan modificarlas ni aplicarles bloqueos hasta que finalice la transacción actual.

- **SELECT ... FOR SHARE:** Nadie puede cambiar la fila hasta que tu hagas commit, con un select normal si (dato antiguo).
- **LOCK IN SHARE MODE:** Bloqueo compartido que asegura que los registros leídos no sean alterados por otros hasta el cierre de la sesión.
- **innodb_deadlock_detect:** Variable para tomar decisiones en caso de interbloqueos
 - La base de datos information_schema, contiene el número de interbloqueos
 - La base de datos performance_schema, contiene información sobre las transacciones.
- **SHOW ENGINE INNODB STATUS:** causa del último interbloqueo.
- **SET TRANSACTION ISOLATION LEVEL [READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE];** Tipos de bloqueos:
- **IX:** Una actualización de una fila normal, sin que implique cambios completos en la tabla.
- **IS:** Simplemente para que no te borren la tabla al completo mientras la lees (pueden seguir modificando filas)
- **S (Shared):** Un select for share por ejemplo que evita que alguien te modifique el dato mientras lo lees (hasta que no hagas commit)
- **X (Exclusive):** Cambios completos de la tabla, como ALTER TABLE o DROP TABLE, no compatibles con nada

0.4.2. GUIÓN PRÁCTICA 4:

0.4.3. Ejercicio: Análisis de Bloqueos en el Motor Transaccional

Listing 2: Monitorización de bloqueos en tiempo real

```
-- 1. Iniciar transaccion y verificar que no hay bloqueos previos
BEGIN;
SELECT * FROM performance_schema.data_locks; -- Resultado: Empty
      set

-- 2. Realizar insercion (Nota: las cadenas requieren comillas
      simples)
INSERT INTO cliente VALUES (1, 'daniel');

-- 3. Consultar metadatos de bloqueo tras la insercion
SELECT ENGINE, OBJECT_SCHEMA, OBJECT_NAME, LOCK_TYPE, LOCK_MODE,
      LOCK_STATUS
FROM performance_schema.data_locks;

-- 4. Salida
```



```

SELECT      ENGINE,      OBJECT_SCHEMA,      OBJECT_NAME,
      LOCK_TYPE,      LOCK_MODE,      LOCK_STATUS,      LOCK_DATA
FROM performance_schema.data_locks;
+-----+-----+-----+-----+-----+
| ENGINE | OBJECT_SCHEMA | LOCK_TYPE | LOCK_MODE | LOCK_STATUS |
+-----+-----+-----+-----+-----+
| INNODB | PR1           | TABLE   | IX        | GRANTED     |
+-----+-----+-----+-----+-----+

```

Resultado

Cuando se esta escribiendo el dato, mysql crea un bloqueo para indicar que se esta realizando una escritura en la tabla 'clientes' para informar al resto de sesiones.

Listing 3: Bloqueos generados por eliminacion y actualizacion

```

-- 10. Borrar fila con nro = 30
BEGIN;
DELETE FROM cliente WHERE nro = 30;

-- 11. Comprobar estado de los bloqueos
SELECT ENGINE, OBJECT_NAME, LOCK_TYPE, LOCK_MODE, LOCK_STATUS,
      LOCK_DATA
FROM performance_schema.data_locks;
-- Salida:
-- TABLE | IX | GRANTED | NULL
-- RECORD | X,REC_NOT_GAP | GRANTED | 30

-- 12. Confirmar y abrir nueva transaccion
COMMIT;
BEGIN;

-- 13. Actualizar fila con nro = 50
UPDATE cliente SET nombre = 'L.Cincuenta' WHERE nro = 50;

-- 14. Comprobar nuevamente los bloqueos
SELECT ENGINE, OBJECT_NAME, LOCK_TYPE, LOCK_MODE, LOCK_STATUS,
      LOCK_DATA
FROM performance_schema.data_locks;
-- Salida:
-- TABLE | IX | GRANTED | NULL
-- RECORD | X,REC_NOT_GAP | GRANTED | 50

-- 15. Finalizar transaccion
COMMIT;

```

Resultado y Justificación

En ambos casos el SGBD registra un bloqueo de intención (IX) sobre la tabla y un bloqueo exclusivo (X) sobre la fila

Paso 1: Sesión 1 - Inserción de 'Ana':

```
BEGIN;  
INSERT INTO cliente VALUES (2, 'Ana');  
SELECT * FROM cliente; -- Se visualizan 3 filas (incluida  
                        'Ana')
```

Paso 2: Sesión 2 - Consulta de control:

```
BEGIN;  
SELECT * FROM cliente; -- Se visualizan 2 filas ('Ana' no  
                        aparece)
```

La Sesión 2 no ve a 'Ana' porque MySQL utiliza por defecto el nivel de aislamiento REPEATABLE READ. Este garantiza que una transacción no lea datos "sucios" (no confirmados) de otras sesiones, manteniendo la consistencia.

Paso 3: Sesión 2 - Inserción de 'Manuel' y verificación cruzada:

```
-- En Sesión 2:  
INSERT INTO cliente VALUES (3, 'Manuel');  
SELECT * FROM cliente; -- Ve a 'Manuel' pero no a 'Ana'  
  
-- En Sesión 1:  
SELECT * FROM cliente; -- Ve a 'Ana' pero no a 'Manuel'
```

Conclusión

Cada sesión trabaja sobre su propia "vista".

0.4.4. Ejercicio: Conflicto de Bloqueos y Lock Wait Timeout

Paso 4: Sesión 2 - Intento de inserción duplicada:

```
-- Intentando insertar el mismo PK (nro=2) que la Sesión 1  
    tiene pendiente  
INSERT INTO cliente VALUES (2, 'Ana');  
-- Resultado tras la espera:  
-- ERROR 1205 (HY000): Lock wait timeout exceeded; try  
    restarting transaction
```

Justificación

El error ocurre porque la Sesión 1 mantiene un bloqueo exclusivo (X) sobre el registro con nro=2. La Sesión 2 intenta adquirir un bloqueo para insertar la misma clave primaria y entra en estado WAITING. Al no liberarse el recurso antes del tiempo límite configurado (*innodb_lock_wait_timeout*), MySQL cancela la instrucción para evitar un bloqueo indefinido.

Paso 5: Comparación de contenido final:

```
-- Sesión 1 ve: nro 1, 2 (Ana), 50
-- Sesión 2 ve: nro 1, 3 (Manuel), 50
```

Conclusión sobre Visibilidad

Ambas sesiones **no ven lo mismo**. Cada una visualiza sus propios cambios no confirmados y los datos previos al inicio de su transacción.

Paso 6: Modificación del tiempo de espera (Timeout):

```
-- Consultar valor por defecto (50 segundos)
SHOW VARIABLES LIKE '%innodb_lock_wait%';

-- Reducir el timeout a 5 segundos para agilizar pruebas
SET innodb_lock_wait_timeout = 5;
```

Paso 7: Sesión 2 - Reintentos de inserción:

```
-- Primer intento: Falla tras 5 segundos por el bloqueo de
    Sesión 1
INSERT INTO cliente VALUES (2, 'Ana');
-- ERROR 1205 (HY000): Lock wait timeout exceeded
```

Paso 8: Sesión 1 - Liberación de recursos:

```
-- Deshacer cambios y liberar bloqueos
ROLLBACK;
SELECT * FROM cliente; -- El nro 2 ya no existe
```

Paso 9: Sesión 2 - Inserción exitosa:

```
-- Segundo intento tras el ROLLBACK de la Sesión 1
INSERT INTO cliente VALUES (2, 'Ana');
-- Query OK, 1 row affected (0,00 sec)
```

Interpretación Final

La reducción del `innodb_lock_wait_timeout` permite que la Sesión 2 detecte el conflicto mucho más rápido. Una vez que la Sesión 1 ejecuta el `ROLLBACK`, el bloqueo exclusivo (X) sobre el registro `nro=2` desaparece, permitiendo que la Sesión 2 tome posesión del recurso e inserte el dato satisfactoriamente.

Para modificar el comportamiento del aislamiento y permitir lecturas de datos no confirmados, se ejecuta:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

Lecturas Sucias

Bajo este modo, desaparece el aislamiento entre transacciones para las operaciones de lectura. Las implicaciones principales serían:

- **Lectura Sucia:** La Sesión 2 vería los INSERT de la Sesión 1 antes del COMMIT.
- **Persistencia de Bloqueos:** A pesar de la visibilidad, los bloqueos de escritura (X) persisten. Si la Sesión 2 intenta modificar o insertar la misma clave que la Sesión 1, el Lock wait timeout seguiría produciéndose.

0.5. Tema 4: Recuperación de la base de datos

0.5.1. Copias de seguridad:

```
[SELECT __/TABLE ___] INTO OUTFILE __ FIELDS TERMINATED BY ','  
OPTIONALLY ENCLOSED BY '"' LINES TERMINATED '\n' [FROM __]
```

Copias lógicas: `mysqldump [opciones]`

Generan archivos .sql que permiten recrear la base de datos.

Casos concretos:

- De una base de datos concreta: `mysqldump db_name [tbl_name]`
- Base de datos completa: `mysqldump --databases db_name`
- Todas las bases de datos: `mysqldump --all-databases`

Algunas opciones de ejecución de *squidump*:

- `-flush-logs`
- `-no-autocommit`
- `-user` `-password`
- `-source-data`: escribe el nombre del log binario de salida
- `-lock-tables`

Ejemplo concreto de uso: `mysqldump --all-databases`

`-user ABDuser -password ABDpwd >all_databases.sql`

Copias incrementales:

- Registros binarios: archivos que registran todas las operaciones DML y DDL realizadas en una base de datos.
- Ver el contenido: `mysqlbinlog <log_file>`
- Tras hacer una copia completa `mysqldump...`, se procederán a realizar copias incrementales sobre esa copia completa (forzar la escritura del log binario y abrir uno nuevo): `mysqladmin flush-logs`

- Para recuperar la base de datos con un proceso de *Point-in-time*:

1. `sudo mysql <backup_completo.sql`
2. `sudo mysqlbinlog <log>`

0.5.2. GUIÓN PRÁCTICA 5:

Creación de copia completa de PR1:

```
mysqldump --source-data --databases PR1 --user root --password >copia-bd-pr1.sql
```

- LOCK TABLES: usado antes de insertar elementos o alterar una tabla.

```
LOCK TABLES '<nombre_tabla>' WRITE
```

- UNLOCK TABLES: usado tras insertar elementos.

Recuperación de PR1 tras borrado de los datos:

```
mysql -u root -p PR1 <copia-bd-pr1.sql
```

Comprobar el valor de la variable 'secure_file_priv':

```
-- Permite ver desde que directorio se pueden escribir/recuperar
las copias
show variables like 'secure_file_priv'
secure_file_priv | /var/lib/mysql-files/
```

Crear copia de los datos de una tabla y recuperarlos despues:

```
-- Permite copiar los datos de una tabla
SELECT * FROM cliente INTO OUTFILE '/var/lib/mysql-files/
cliente_copia.csv' FIELDS TERMINATED BY ',' LINES
TERMINATED BY '\n';
-- Permite restaurar los datos de una tabla (necesita de la
estructura de la tabla, solo recupera los datos)
Load data infile '/var/lib/mysql-files/cliente_copia.csv' into
table cliente fields terminated by ',' lines terminated by '
\n';
```

Comprobación de logs binarios:

```
show variables like 'log_bin%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_bin       | ON    |
| log_bin_basename | /var/lib/mysql/binlog |
+-----+-----+
```

Copia completa de la base de datos:

```
-- single-transaction nos asegura que sea en caliente, --source
-data nos asegura que se escriba el log binario y que se
cree uno nuevo.
sudo mysqldump --all-databases --single-transaction --source-
data=2 --flush-logs -u root -p > /home/usuario/
backup_online_completo.sql
```

```
-- Tras crear una nueva base de datos con una tabla y algunos
    valores: (esto creara un nuevo archivo binario con los
    cambios (copia incremental))
mysqladmin -u root -p flush-logs

-- Tras dropear todas las bases de datos, para recuperarlas:
mysql < backup_online_completo.sql
mysqlbinlog /var/lib/mysql/binlog.000133 | mysql -u root -p
```

0.5.3. Plan de copias de seguridad de una empresa (supuesto)

1. Identificación de Aspectos Teóricos y Clasificación de Datos El plan implementa una estrategia de **recuperación híbrida** que garantiza la durabilidad ante fallos de instancia o disco.

- **Clasificación de Datos:** Se aplica un análisis de **importancia** y **volatilidad**.
 - *Datos Críticos/Estáticos:* Información personal de empleados (baja variabilidad).
 - *Datos Críticos/Dinámicos:* Inversiones y bienes con alta variabilidad (alta volatilidad).
- **Adecuación:** La frecuencia semanal para la copia completa y horaria para logs es correcta para minimizar el punto de recuperación en activos volátiles, evitando la pérdida de transacciones financieras.

2. Utilidades de MySQL y Comandos de Ejecución Para llevar a la práctica el plan de la empresa, se proponen las siguientes herramientas del SGBD:

Paso del Plan	Utilidad	Comando Sugerido
Copia Completa	mysqldump	mysqldump -all-databases -single-transaction -source-data=2 -flush-logs >full.sql
Copia Incremental	mysqladmin	mysqladmin flush-logs
Copia de Logs	SO / Bash	cp /var/lib/mysql/binlog.* /backup/remote/
Restauración	mysql	mysql <backup_completo.sql
Aplicación Logs	mysqlbinlog	mysqlbinlog binlog.0000X mysql -u root -p

3. Cumplimiento del ENS (Esquema Nacional de Seguridad) El plan cumple con los pilares de **disponibilidad, integridad y confidencialidad**:

- **Regla 3-2-1:** Cumplida mediante almacenamiento local externo y remoto cifrado en la nube.
- **Cifrado:** Obligatorio para datos personales de empleados en tránsito y reposo.
- **Auditoría:** Registro de copias y revisiones mensuales de infraestructura.

4. Plan Definitivo y Propuesta de Mejora Se modifica la propuesta inicial para optimizar la recuperación en escenarios de alta concurrencia:

1. **Frecuencia de Logs:** Reducción a **30 minutos**. *Justificación:* La volatilidad de los bienes exige un menor tiempo de pérdida de datos.
2. **Copia de Índices:** Opcional. Se decide reconstruirlos tras la carga de datos para reducir el tamaño del backup.
3. **Puntos de Validación (Checkpoints):** Forzar el volcado de búferes a disco antes de cada rotación de log para garantizar consistencia.
4. **Método:** Uso de `-single-transaction` para evitar bloqueos de escritura y mantener la disponibilidad de las sedes.

0.6. Tema 5: Seguridad y privacidad en bases de datos

0.6.1. Creación de usuarios

```
CREATE USER [IF NOT EXISTS] -- if not exists asegura que no de
    error en caso de que ya exista ese nombre de usuario
user [auth_option] [, user [auth_option]] ... -- password por
    ejemplo es una auth_option
DEFAULT ROLE role [, role] ... -- define rol de usuario para
    definir permisos a nivel de cuenta (MAC)
[REQUIRE {NONE | tls_option [[AND] tls_option] ...}] --
    requerir certificado TLS
[WITH resource_option [resource_option] ...] -- metricas limite
    de usuario
[password_option | lock_option] -- caducidad contrasena,
    cuenta de usuario bloqueada,...
[COMMENT 'comment_string' | ATTRIBUTE 'json_object'] --
    descripcion
```

Ejemplo:

```
CREATE USER IF NOT EXISTS 'analista_broker'@'localhost'
    IDENTIFIED WITH caching_sha2_password BY 'Password_Segura_2026'
    ,
    DEFAULT ROLE 'consultor_datos', 'auditor'
    REQUIRE SSL
    WITH MAX_QUERIES_PER_HOUR 500
        MAX_UPDATES_PER_HOUR 50
        MAX_CONNECTIONS_PER_HOUR 10
        MAX_USER_CONNECTIONS 2
    PASSWORD EXPIRE INTERVAL 90 DAY
    ACCOUNT UNLOCK
    COMMENT 'Usuario del departamento de facturacion para sedes
        locales';
```

Creación sencilla de usuario desde cuenta root en localhost:

```
CREATE USER 'name'@'localhost' IDENTIFIED BY 'password';
```

Cambio de password de un usuario:

```
ALTER USER myuser@localhost IDENTIFIED BY 'newPass';
```

Información sobre los usuarios del sistema

```
SELECT user, host from mysql.user;
```

0.6.2. Concesión de permisos

GRANT

```
-- Permisos de INSERT a toda la tabla facturacion.licencias y
-- de SELECT solo a las 2 columnas especificadas a usuario.
GRANT INSERT, SELECT (id_contenido, fecha)
ON facturacion.licencias
TO 'usuario'@'%'
WITH GRANT OPTION;
```

REVOKE

```
-- Revocar permisos de INSERT a toda la tabla facturacion.
-- licencias y de SELECT solo a las 2 columnas especificadas a
-- usuario.
REVOKE INSERT, SELECT (id_contenido, fecha), GRANT OPTION
ON facturacion.licencias
FROM 'broker_local'@'%';
```

```
-- Revocar todos los permisos sobre una tabla de un empleado.
REVOKE ALL PRIVILEGES, GRANT OPTION ON <table> FROM <user>;
```

PROPIEDADES DE UNA CUENTA

```
-- Ver permisos asociados a un usuario.
SHOW GRANTS FOR 'bob'@'pc84.example.com';
```

0.6.3. ROLES

```
-- Creación del rol
create role <role_name>;
-- Creación del usuario
create user <user_name> @localhost identified by '...';
-- Dar los permisos acordes al rol
grant select, insert, update, delete on <databases>.<table> to
<role_name>;
-- asignar usuario al rol
grant <role_name> to <user_name>@localhost;
-- asignarlo por defecto
SET DEFAULT ROLE <role_name> TO <user_name>@localhost;
```

El usuario puede ver el rol asociado y los permisos que posee con los comandos:

```
show grants;
select current_role();
```


0.6.4. BLOQUEO DE CUENTAS DE USUARIO

```
ALTER USER usuario ACCOUNT LOCK;
```

0.6.5. ACTIVAR FICHEROS DE LOG PARA AUDITORÍA

```
set global general_log=ON;
```

0.6.6. CIFRADO DE DATOS

```
INSERT INTO t VALUES(1,AES_ENCRYPT('text', UNHEX(SHA2('My  
secretphrase',512))));
```

0.6.7. GUIÓN PRACTICA 6:

Comprobación de mecanismo de autenticación de los distintos usuarios:

```
select user, host, plugin from mysql.user;  
-- auth_socket valida que usuario que intenta entrar a la base  
  de datos sea el mismo que el del sistema  
-- caching_sha2_password es la contrasena comun
```

Comprobación de cuentas creadas por defecto:

```
root: administracion global, configuracion de otros usuarios y  
      gestion.  
mysql.sys: ejecucion de objetos almacenados que requieren  
           acceso al sistema de forma segura  
mysql.session: uso de plugins y componentes  
mysql.infoschema: consulta metadatos del sistema  
-- en mysql.db se definen permisos de usuario sobre bases de  
  datos especificas
```

0.6.8. GUIÓN PRÁCTICA 7:

Ejercicio de propagación de permisos con DAC:

```
GRANT SELECT, INSERT, DELETE ON PruebaSeg.Empleado to 'User2';  
GRANT SELECT ON PruebaSeg.Empleado to 'User3' WITH GRANT OPTION  
;  
GRANT SELECT ON PruebaSeg.Departamento to 'User3' WITH GRANT  
  OPTION;  
(User3) GRANT SELECT ON PruebaSeg.Empleado to 'User4';  
REVOKE SELECT ON PruebaSeg.Empleado WITH GRANT OPTION FROM '  
  User3';  
CREATE VIEW vista_depno5 AS SELECT nombre, fnac, dir from  
  empleado where depno=5;  
GRANT SELECT ON vista_depno5 TO 'User3' WITH GRANT OPTION;  
GRANT UPDATE sueldo ON PruebaSeg.Empleado TO 'User4';
```

0.6.9. Conexiones desde el exterior

```
create user userconectar@'%' identified by 'password';  
mysql> grant all privileges on database.* to userconectar;
```